Robotik Mühendisliği Serisi — Cilt 4A

Kontrol Temelleri

PID, frekans tanım, durum uzayı, hesaplanmış tork, empedans

Türkçe–İngilizce karma konu anlatımı
teori · sayısal örnekler · çalışan Python ve C++ kodları

Odak: bacaklı ve humanoid robotlar

31 Ekim 2026

İçindekiler

Bu Cilt Hakkında **1**

1 İkinci Dereceden Sistemler **2**

2 PID ve Gerçek Dünya Tuzakları **4**

3 Frekans Tanım: Bode, Faz Payı, Gecikme **8**

4 Durum Uzayı, Kutup Yerleştirme, Gözlemci, Ayrıklaştırma **10**

4.1 Durum uzayı ve kontrol edilebilirlik 10

4.2 Kutup yerleştirme 10

4.3 Ayrıklaştırma 10

4.4 Gözlemci 11

5 Yerçekimi Kompanzasyonu ve Hesaplanmış Tork **13**

6 Empedans Kontrolü **15**

7 Cilt 4B'ye Geçiş Haritası **17**

7.1 4A ile 4B arasındaki köprüler 17

7.2 Staj döneminde 4B ile kâğıt-kalem 17

8 Sorular **18**

9 Cevaplar **20**

Bu Cilt Hakkında

Cilt 4 iki kitaptan oluşuyor:

- **Cilt 4A (bu belge): Kontrol Temelleri.** Tedrake'in *Underactuated Robotics* notlarının "bildiğini varsaydığı" klasik kontrol: ikinci dereceden sistemler, PID ve gerçek uygulamadaki tuzakları (doyma, integral birikmesi, türev gürültüsü), frekans tanım ve gecikme, durum uzayı, kutup yerleştirme, gözlemci, ayrıklaştırma, hesaplanmış tork ve empedans kontrolü.
- **Cilt 4B: Underactuated Robotics Çalışma Rehberi** (daha önce verdiğim 163 sayfalık rehber): sarkaçtan funnel'lara, LQR, Lyapunov, trajectory optimization, MPC, temas ve öğrenmeye kadar ileri kontrol.

Önce 4A, sonra 4B'nin çekirdek bölümleri (2, 5, 7, 8, 9, 10, 11). 4A'daki örnek sistem, Cilt 3'teki servo + uyluk modeli ($J = 0.0116$ kg·m^2) ve Cilt 1'deki 2-uzuvlu bacak.

Staj dönemi için: Telefon sürümünde (Kitapçık 4) kodlar yok; metin, sayılar, şekiller ve 24 soru-cevap var. Hepsi elle çözülebilir.

Bölüm	Konu	Kod
1	İkinci dereceden sistemler	`e1_second_order.py`
2	PID: doyma, anti-windup, türev filtresi	`e2_pid.py/.cpp`
3	Frekans tanım: Bode, faz payı, gecikme	`e3_freq.py`
4	Durum uzayı, kutup yerleştirme, gözlemci, ayrıklaştırma	`e4_statespace.py`, `e4_zoh.cpp`
5	Yerçekimi kompanzasyonu ve hesaplanmış tork	`e5_computed_torque.py`
6	Empedans kontrolü	`e6_impedance.py`
7	Cilt 4B'ye geçiş haritası	–
8–9	Sorular ve cevaplar	–

Bölüm 1

İkinci Dereceden Sistemler

Bir eklemi PD ile kontrol edince ($J\ddot{\theta} = K_p(\theta_d - \theta) - K_d\dot{\theta}$) kapalı çevrim

$$\ddot{\theta} + 2\zeta\omega_n\dot{\theta} + \omega_n^2\theta = \omega_n^2\theta_d, \qquad \omega_n = \sqrt{K_p/J}, \qquad \zeta = \frac{K_d}{2\sqrt{K_pJ}}$$

olur. Kontrol mühendisliğinin "dili" bu iki sayıdır: ω_n (doğal frekans, ne kadar hızlı) ve ζ (sönüm oranı, ne kadar salınımlı). Birim basamak yanıtı için kullanışlı formüller ($0 < \zeta < 1$):

$$\text{aşım } M_p = e^{-\pi\zeta/\sqrt{1-\zeta^2}}, \qquad t_{\text{yerleşme}}(\%2) \approx \frac{4}{\zeta\omega_n},$$

$$\text{kutuplar } s = -\zeta\omega_n \pm j\omega_n\sqrt{1-\zeta^2}.$$

Tasarımın tersi: istenen ω_n, ζ'dan $K_p = J\omega_n^2$, $K_d = 2\zeta\omega_n J$. **Önemli:** kazançlar ataletle ölçeklenir. Aynı K_p hafif bir uzuvda hızlı, ağır bir uzuvda yavaş bir yanıt verir; bu yüzden kazançları "atalet başına" düşünmek (veya Bölüm 5'teki gibi modelle normalize etmek) iyi bir alışkanlıktır.

Sayılar: servo + uyluk, $J = 0.0116$

K_p	K_d	ω_n	ζ	M_p formül	M_p sim.	yükselme	t_s formül	t_s sim.
2	0.05	13.1	0.16	59.3%	59.3%	89 ms	1856 ms	1752 ms
2	0.15	13.1	0.49	16.9%	16.9%	124 ms	619 ms	620 ms
2	0.30	13.1	0.98	0.0%	0.0%	250 ms	309 ms	431 ms
8	0.30	26.3	0.49	16.9%	16.9%	62 ms	309 ms	310 ms

Aşım formülü kesin. Yerleşme formülü ζ küçükken iyi, kritik sönüme yakınken ($\zeta \approx 1$) kötü bir yaklaşım (309 vs 431 ms): her "kural"ın geçerlilik aralığını bil. Son iki satır: K_p dört katına çıkınca ve ζ korunca yanıt iki kat hızlanıyor, şekli aynı kalıyor.

```python
import numpy as np
from scipy.signal import lti, step

# İkinci dereceden sistem: ẍ + 2ζω_n ẋ + ω_n² x = ω_n² u   (birim basamak)
# Örnek: PD kontrollü bir eklem  J θ¨ = Kp(θd - θ) - Kd θ˙  ->  ω_n = sqrt(Kp/J), ζ = Kd / (2 sqrt(Kp J))
J = 0.0116                                  # Cilt 3: servo+uyluk etkin ataleti [kg·m²]
print(f"{'Kp':>6} {'Kd':>6} | {'ω_n':>6} {'ζ':>5} | aşım formül/sim | yükselme(10-90%) | yerleşme(%2) formül 4/(ζω_n) / sim")
for Kp, Kd in [(2.0, 0.05), (2.0, 0.15), (2.0, 0.3), (8.0, 0.3)]:
    wn = np.sqrt(Kp/J); z = Kd/(2*np.sqrt(Kp*J))
    sys = lti([wn**2], [1, 2*z*wn, wn**2]); t, y = step(sys, T=np.linspace(0, 2, 20001))
    Mp = np.exp(-np.pi*z/np.sqrt(1 - z*z))*100 if z < 1 else 0.0
    os_sim = max(0, (y.max() - 1)*100)
    tr = t[np.argmax(y >= 0.9)] - t[np.argmax(y >= 0.1)]
    out = np.where(np.abs(y - 1) > 0.02)[0]; ts = t[out[-1] + 1] if len(out) else 0
    print(f"{Kp:6.1f} {Kd:6.2f} | {wn:6.2f} {z:5.2f} | {Mp:5.1f}% / {os_sim:5.1f}% | {tr*1000:7.1f} ms | {4/(z*wn)*1000:7.1f} ms / {ts*1000:7.1f} ms")
```

Çıktı:

```
  Kp     Kd |    ω_n     ζ | aşım formül/sim | yükselme(10-90%) | yerleşme(%2) formül 4/(ζω_n) / sim
 2.0   0.05 |  13.13  0.16 |  59.3% /  59.3% |    88.8 ms       |  1856.0 ms /  1751.9 ms
 2.0   0.15 |  13.13  0.49 |  16.9% /  16.9% |   123.6 ms       |   618.7 ms /   620.1 ms
 2.0   0.30 |  13.13  0.98 |   0.0% /   0.0% |   250.1 ms       |   309.3 ms /   431.2 ms
 8.0   0.30 |  26.26  0.49 |  16.9% /  16.9% |    61.8 ms       |   309.3 ms /   310.1 ms
```

Bölüm 2

PID ve Gerçek Dünya Tuzakları

$u = K_p e + K_i \int e\,dt + K_d \dot{e}$. Kâğıtta basit; gerçek bir eklemde dört sorun çıkar:

1. **Kalıcı hata (P/PD):** sabit bir yük (yerçekimi) varsa PD bir hata ile dengeye gelir: $K_p e = \tau_{\text{yük}} \Rightarrow e = \tau_{\text{yük}}/K_p$. İntegral terimi bu hatayı sıfırlar (veya modelle kompanze et: Bölüm 5).
2. **İntegral birikmesi (windup):** aktüatör doymadayken hata büyük kalır ve integral şişer; hedefe gelindiğinde biriken integral boşalana kadar sistem hedefi aşar. Çare (**anti-windup**): *kırpma* (doymadayken ve hata doymayı büyütüyorsa integrali durdur) veya *geri hesaplama* (doyma farkını $k_b(u - u_{\text{doymamış}})$ ile integrale geri besle).
3. **Türev tekmesi:** hedef basamakla değişince $\dot{e}$ anlık olarak sonsuz olur. Türevi hatadan değil *ölçümden* al: $-\dot{y}$ (hedef sabitken aynı şey).
4. **Türev gürültüsü:** Cilt 3 Bölüm 4: ölçüm gürültüsü türevde $1/\Delta t$ ile büyür. Türeve bir alçak geçiren filtre ekle: $d_k = a\,d_{k-1} + (1-a)\,\dot{y}_{ham}$, $a = \tau_f/(\tau_f + \Delta t)$.

Sayılar: yerçekimi yüklü eklem, tork sınırı ± 1 N·m

$J = 0.0116$, $mg\ell_c = 0.6$; hedef 1.2 rad (orada yerçekimi torku 0.559 N·m); $K_p = 8$, $K_d = 0.4$, $K_i = 20$; 1 kHz.

Kontrolcü	aşım	3 s sonra hata	yerleşme (%2)
PD	hedefe ulaşmıyor	0.068 rad (3.9°)	–
PID, anti-windup yok	**22.6%**	0.0001	1.13 s
PID, kırpma	0	0.0000	**0.24 s**
PID, geri hesaplama ($k_b = 20$)	0	0.0002	1.25 s

PD'nin kalıcı hatası elle: $0.559/8 = 0.070$ rad (simülasyon 0.068; yerçekimi torku hatalı açıda biraz daha küçük). Anti-windup'sız PID hedefi %22.6 aşıyor; kırpma ile aşım yok ve 0.24 s'de yerleşiyor. Geri hesaplama burada yavaş kaldı: kazancı k_b ayara bağlı; "her zaman en iyi" bir yöntem yok, sisteminde dene. Python ve C++ aynı sayıları verdi. **Gürültü** ($\sigma = 0.002$ rad, $\approx 0.1°$): türev filtresi yokken torkun RMS dalgalanması 0.615 N·m (sınırın %60'ı!), $\tau_f = 5$ ms ile 0.154, 20 ms ile 0.054. Bedeli faz gecikmesi (Bölüm 3).

Python

```python
import numpy as np

# Tork kontrollü bir eklem: J θ¨ = u - b θ˙ - m g l_c sin θ,  |u| <= U_MAX  (tork doyması)
J, b, mgl, U_MAX = 0.0116, 0.01, 0.6, 1.0
dt, T = 0.001, 3.0

class PID:
    """Ayrık PID: türev ÖLÇÜMDEN (türev tekmesi yok), 1. derece türev filtresi, iki anti-windup seçeneği."""
    def __init__(self, kp, ki, kd, dt, u_max, tf=0.01, aw="none", kb=None):
        self.kp, self.ki, self.kd, self.dt, self.u_max, self.aw = kp, ki, kd, dt, u_max, aw
```

```
        self.a = tf/(tf + dt)                       # türev filtresi katsayısı (zaman sabiti tf)
        self.kb = kb if kb is not None else 1.0/max(kp, 1e-9)*ki   # back-calculation kazancı
        self.i = 0.0; self.d = 0.0; self.y_prev = None
    def update(self, r, y):
        e = r - y
        if self.y_prev is None: self.y_prev = y
        d_raw = -(y - self.y_prev)/self.dt; self.y_prev = y             # -dy/dt (r sabitken = de/dt)
        self.d = self.a*self.d + (1 - self.a)*d_raw
        u_unsat = self.kp*e + self.i + self.kd*self.d
        u = float(np.clip(u_unsat, -self.u_max, self.u_max))
        if self.aw == "clamp":                                          # doymadaysa ve hata doymayı
        büyütüyorsa integrali durdur
            if not (u != u_unsat and np.sign(e) == np.sign(u_unsat)): self.i += self.ki*e*self.dt
        elif self.aw == "back":                                         # geri hesaplama: doyma farkı
        integrali geri çeker
            self.i += (self.ki*e + self.kb*(u - u_unsat))*self.dt
        else:
            self.i += self.ki*e*self.dt
        return u

def run(ctrl, r=1.2, noise=0.0, seed=0):
    rng = np.random.default_rng(seed); th = w = 0.0; Y = []; U = []
    for k in range(int(T/dt)):
        y = th + noise*rng.standard_normal()
        u = ctrl.update(r, y); w += dt*(u - b*w - mgl*np.sin(th))/J; th += dt*w; Y.append(th);
        U.append(u)
    Y = np.array(Y); U = np.array(U)
    out = np.where(np.abs(Y - r) > 0.02*r)[0]; ts = (out[-1] + 1)*dt if len(out) else 0
    return Y, U, (Y.max() - r)/r*100, abs(Y[-1] - r), ts

print("hedef 1.2 rad (yerçekimi torku sin(1.2)·0.6 = %.3f N·m), tork sınırı ±%.1f N·m" %
(mgl*np.sin(1.2), U_MAX))
cases = [("PD (Kp =8, Kd =0.4)", PID(8, 0, 0.4, dt, U_MAX)),
         ("PID, anti-windup YOK", PID(8, 20, 0.4, dt, U_MAX)),
         ("PID, kırpma (clamp)", PID(8, 20, 0.4, dt, U_MAX, aw="clamp")),
         ("PID, geri hesaplama", PID(8, 20, 0.4, dt, U_MAX, aw="back", kb=20.0))]
res = {}
for name, c in cases:
    Y, U, os_, ess, ts = run(c); res[name] = (Y, U)
    print(f"  {name:>22}: aşım {os_:6.1f}%, 3 s sonra hata {ess:.4f} rad, yerleşme (%2) {ts:.2f} s,
    doymada geçen süre {np.mean(np.abs(U) >= U_MAX)*T:.2f} s")

# Ölçüm gürültüsü ve türev filtresi
for tf in [0.0, 0.005, 0.02]:
    c = PID(8, 20, 0.4, dt, U_MAX, tf=tf, aw="clamp"); Y, U, *_ = run(c, noise=0.002)
    print(f"gürültü σ =0.002 rad, türev filtresi τ_f ={tf*1000:4.0f} ms: tork RMS dalgalanması (son 1
    s) {np.std(U[-1000:]):.3f} N·m")
```

Çıktı:

```
hedef 1.2 rad (yerçekimi torku sin(1.2)·0.6 = 0.559 N·m), tork sınırı ±1.0 N·m
       PD (Kp =8, Kd =0.4): aşım   -5.4%, 3 s sonra hata 0.0679 rad, yerleşme (%2) 3.00 s, doymada geçen süre 0.15 s
    PID, anti-windup YOK: aşım   22.6%, 3 s sonra hata 0.0001 rad, yerleşme (%2) 1.13 s, doymada geçen süre 0.19 s
     PID, kırpma (clamp): aşım   -0.0%, 3 s sonra hata 0.0000 rad, yerleşme (%2) 0.24 s, doymada geçen süre 0.13 s
     PID, geri hesaplama: aşım   -0.0%, 3 s sonra hata 0.0002 rad, yerleşme (%2) 1.25 s, doymada geçen süre 0.08 s
gürültü σ =0.002 rad, türev filtresi τ_f =   0 ms: tork RMS dalgalanması (son 1 s) 0.615 N·m
gürültü σ =0.002 rad, türev filtresi τ_f =   5 ms: tork RMS dalgalanması (son 1 s) 0.154 N·m
gürültü σ =0.002 rad, türev filtresi τ_f =  20 ms: tork RMS dalgalanması (son 1 s) 0.054 N·m
```

C++: gerçek zamanlı koda konacak PID sınıfı

```cpp
// e2_pid.cpp — gerçek zamanlı koda konacak biçimde PID sınıfı (Python e2_pid.py ile aynı algoritma)
#include <algorithm>
#include <cmath>
#include <cstdio>

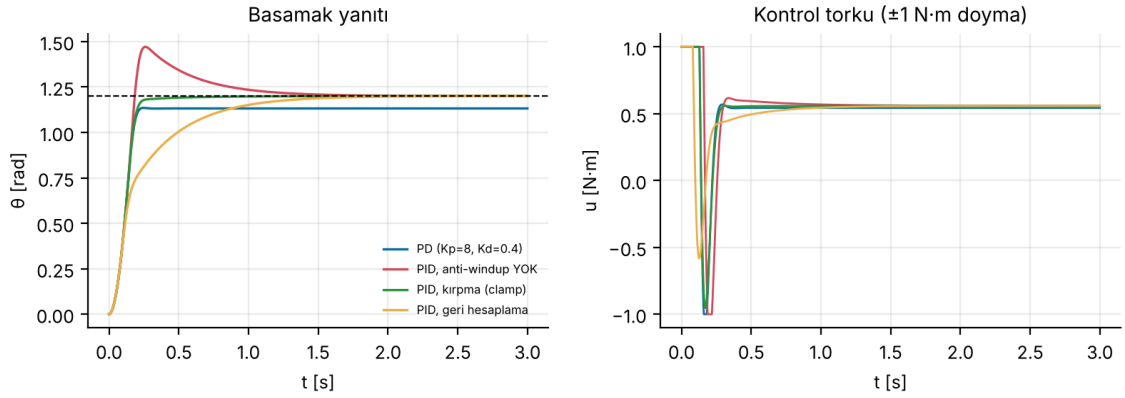
class Pid {
public:
    enum class AntiWindup { None, Clamp, BackCalc };
    Pid(double kp, double ki, double kd, double dt, double u_max, double tf = 0.01,
        AntiWindup aw = AntiWindup::None, double kb = 0.0)
        : kp_(kp), ki_(ki), kd_(kd), dt_(dt), u_max_(u_max), a_(tf / (tf + dt)), aw_(aw), kb_(kb) {}

    double update(double r, double y) {
        const double e = r - y;
        if (first_) { y_prev_ = y; first_ = false; }
        const double d_raw = -(y - y_prev_) / dt_;          // türev ölçümden: hedef basamağında "tekme" yok
        y_prev_ = y;
        d_ = a_ * d_ + (1.0 - a_) * d_raw;                   // 1. derece filtre
        const double u_unsat = kp_ * e + i_ + kd_ * d_;
        const double u = std::clamp(u_unsat, -u_max_, u_max_);
        switch (aw_) {
            case AntiWindup::Clamp:
                if (!(u != u_unsat && (e > 0) == (u_unsat > 0))) i_ += ki_ * e * dt_;
                break;
            case AntiWindup::BackCalc: i_ += (ki_ * e + kb_ * (u - u_unsat)) * dt_; break;
            case AntiWindup::None: i_ += ki_ * e * dt_; break;
        }
        return u;
    }
    void reset() { i_ = d_ = 0.0; first_ = true; }
private:
    double kp_, ki_, kd_, dt_, u_max_, a_;
    AntiWindup aw_; double kb_;
    double i_ = 0.0, d_ = 0.0, y_prev_ = 0.0; bool first_ = true;
};

int main() {
    const double J = 0.0116, b = 0.01, mgl = 0.6, dt = 0.001, r = 1.2;
    struct Case { const char* name; Pid pid; };
    Case cases[] = {{"PD", Pid(8, 0, 0.4, dt, 1.0)},
                    {"PID, anti-windup yok", Pid(8, 20, 0.4, dt, 1.0)},
                    {"PID, kırpma", Pid(8, 20, 0.4, dt, 1.0, 0.01, Pid::AntiWindup::Clamp)},
                    {"PID, geri hesaplama", Pid(8, 20, 0.4, dt, 1.0, 0.01, Pid::AntiWindup::BackCalc, 20.0)}};
    for (auto& c : cases) {
        double th = 0, w = 0, ymax = 0; int last_out = 0;
        for (int k = 0; k < 3000; ++k) {
            double u = c.pid.update(r, th);
            w += dt * (u - b * w - mgl * std::sin(th)) / J; th += dt * w;
            ymax = std::max(ymax, th);
            if (std::abs(th - r) > 0.02 * r) last_out = k + 1;
        }
        std::printf("%-22s aşım %6.1f%%, son hata %.4f rad, yerleşme %.2f s\n", c.name, (ymax - r) / r * 100, std::abs(th - r), last_out * dt);
    }
    return 0;
}
```

Çıktı:

```
PD                    aşım   -5.4%, son hata 0.0679 rad, yerleşme 3.00 s
PID, anti-windup yok  aşım   22.6%, son hata 0.0001 rad, yerleşme 1.13 s
PID, kırpma          aşım   -0.0%, son hata 0.0000 rad, yerleşme 0.24 s
PID, geri hesaplama   aşım   -0.0%, son hata 0.0002 rad, yerleşme 1.25 s
```



Şekil 2.1: Sol: dört kontrolcünün basamak yanıtı. Sağ: kontrol torku; anti-windup'sız PID doymada uzun süre kalıp integral biriktiriyor.

C++ notu: `enum class` ve başlatma listesi

`enum class AntiWindup {None, Clamp, BackCalc}` tip güvenli bir seçenek listesi (C'deki `enum`'dan farkı: isimler `AntiWindup::Clamp` diye yazılır, tamsayıya kendiliğinden dönüşmez). Kurucudaki `: kp_(kp), ki_(ki), ...` *başlatma listesi*: üyeler gövde çalışmadan önce kurulur; `const` üyeler ancak böyle başlatılabilir. `reset()` fonksiyonu, kontrolcü yeniden etkinleştirildiğinde eski integralin "tekme" atmasını önler; gerçek sistemlerde mod geçişlerinde mutlaka çağrılır.

Bölüm 3

Frekans Tanım: Bode, Faz Payı, Gecikme

Kapalı çevrimin kararlılığı, *açık çevrim* $L(j\omega) = C(j\omega)P(j\omega)$'nin frekans yanıtından okunabilir:

- **Kazanç kesişimi** ω_c: $|L(j\omega_c)| = 1$. Kabaca kapalı çevrimin bant genişliği.
- **Faz payı** $PM = 180^\circ + \angle L(j\omega_c)$: kararsızlığa kadar ne kadar faz kaybı tolere edilebilir. Tipik hedef 45°–60°. Düşük faz payı = salınımlı yanıt.
- **Kazanç payı:** faz -180°'yi geçtiği frekansta $|L|$'nin 1'in kaç dB altında olduğu.
- **Gecikme:** $e^{-j\omega\tau}$: genliği değiştirmez, fazdan $\omega\tau$ radyan yer. Kesişimde PM kadar faz varsa tolere edilebilecek gecikme $\tau_{max} = PM/\omega_c$. Robotik için en önemli kural budur: **bant genişliğini artırmak gecikme payını azaltır.**

Sayılar: PD kontrollü eklem, $P = 1/(Js^2 + bs)$, türev filtresi 5 ms

K_p	K_d	gecikme	ω_c [rad/s]	faz payı	kazanç payı	kalan gecikme payı
2	0.15	0	17.0	51.7°	∞	53 ms
8	0.30	0	34.8	47.5°	∞	24 ms
8	0.30	10 ms	34.8	27.5°	10.3 dB	14 ms
8	0.30	20 ms	34.8	7.6°	2.7 dB	4 ms
30	0.60	0	70.2	41.7°	∞	10 ms
30	0.60	20 ms	70.2	-38.8°	–	**kararsız**

Elle kontrol: 10 ms gecikme, $\omega_c = 34.8$'de $34.8 \cdot 0.01 = 0.348$ rad $= 19.9^\circ$ yer; $47.5 - 19.9 = 27.6^\circ$. **Hobi servosunun 20 ms'lik komut periyodu** (Cilt 3) tam bu satırlardaki gecikme: dışarıdan servoyla kurulan bir döngü, $K_p = 30$ gibi "sert" bir ayarda kararsız olur. Gecikmesizken de faz payı sonsuz değil: türev filtresi ve sürtünme faz yer.

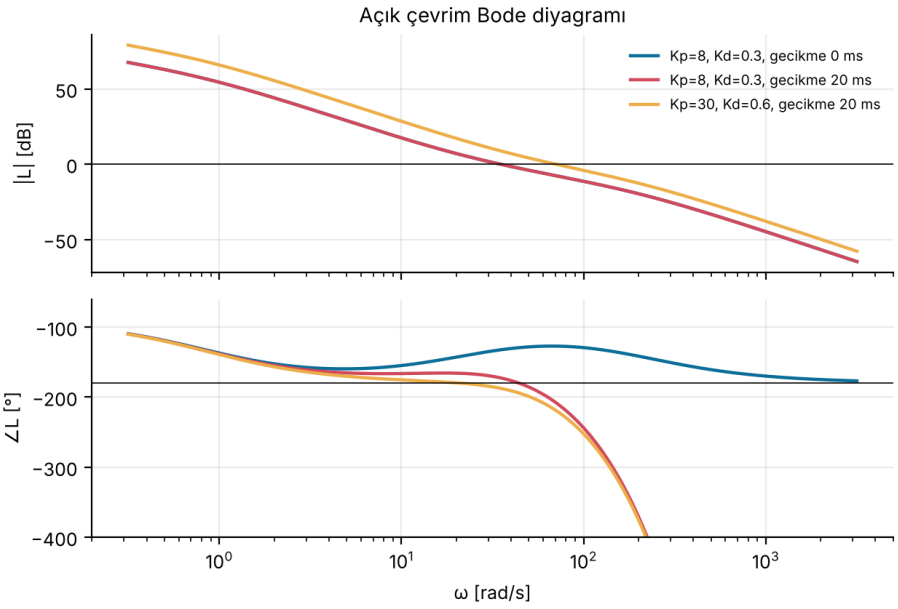
```python
import numpy as np
from scipy.optimize import brentq

# Açık çevrim L(jω) = C(jω) P(jω);  eklem P = 1/(J s² + b s),  PD: C = Kp + Kd s/(τ_f s + 1);  gecikme e^{-sτ}
J, b, tf = 0.0116, 0.01, 0.005
P = lambda s: 1.0/(J*s*s + b*s)
C = lambda s, Kp, Kd: Kp + Kd*s/(tf*s + 1)
def margins(Kp, Kd, delay=0.0):
    L = lambda w: C(1j*w, Kp, Kd)*P(1j*w)*np.exp(-1j*w*delay)
    wc = brentq(lambda w: abs(L(w)) - 1, 1e-2, 1e4)                          # kazanç kesişimi |L| = 1
    pm = 180 + np.degrees(np.unwrap(np.angle(L(np.logspace(-2, np.log10(wc), 5000))))[-1])   # faz SARILMADAN (gecikme -180°'yi aşabilir)
    ws = np.logspace(-1, 4, 20000); ang = np.unwrap(np.angle(L(ws)))
    idx = np.where(ang <= -np.pi)[0]                                         # faz -180°'yi geçtiği yer
    gm = -20*np.log10(abs(L(ws[idx[0]]))) if len(idx) else np.inf
    return wc, pm, gm, np.radians(pm)/wc
print(f"{'Kp':>5} {'Kd':>5} {'gecikme':>8} | ω_c [rad/s]  faz payı  kazanç payı   gecikme payı")
for Kp, Kd, d in [(2, 0.15, 0), (8, 0.3, 0), (8, 0.3, 0.010), (8, 0.3, 0.020), (30, 0.6, 0), (30, 0.6, 0.020)]:
```

```
    wc, pm, gm, dm = margins(Kp, Kd, d)
    gms = "∞" if not np.isfinite(gm) else f"{gm:5.1f} dB"
    print(f"{Kp:5.0f} {Kd:5.2f} {d*1000:6.0f} ms | {wc:9.2f}  {pm:7.1f}°  {gms:>10}   " +
    (f"{dm*1000:7.1f} ms" if pm > 0 else "KARARSIZ (faz payı < 0)"))
print("kural: gecikme τ, kesişim frekansında faz payından ω_c·τ (radyan) yer: 20 ms gecikme, ω_c = 30 rad/s'de 34° demek")
```

Çıktı:

```
   Kp    Kd  gecikme | ω_c [rad/s]  faz payı  kazanç payı   gecikme payı
    2  0.15      0 ms |     17.03     51.7°           ∞      53.0 ms
    8  0.30      0 ms |     34.83     47.5°           ∞      23.8 ms
    8  0.30     10 ms |     34.83     27.5°     10.3 dB      13.8 ms
    8  0.30     20 ms |     34.83      7.6°      2.7 dB       3.8 ms
   30  0.60      0 ms |     70.21     41.7°           ∞      10.4 ms
   30  0.60     20 ms |     70.21    -38.8°    -16.4 dB   KARARSIZ (faz payı < 0)
kural: gecikme τ, kesişim frekansında faz payından ω_c·τ (radyan) yer: 20 ms gecikme, ω_c = 30 rad/s'de 34° demek
```



Şekil 3.1: Açık çevrim Bode diyagramı. Gecikme genliği değiştirmiyor ama fazı yüksek frekanslarda hızla aşağı çekiyor; daha yüksek kazanç kesişimi sağa taşıyıp gecikmenin etkisini büyütüyor.

Örnekleme frekansı kuralı: dijital kontrolcü, sıfır dereceli tutucu yüzünden yaklaşık $\Delta t/2$ gecikme ekler. Kapalı çevrim bant genişliğinin 10–20 katı örnekleme frekansı güvenli bir başlangıçtır (Bölüm 4'te sayısal örnek).

Bölüm 4

Durum Uzayı, Kutup Yerleştirme, Gözlemci, Ayrıklaştırma

4.1 Durum uzayı ve kontrol edilebilirlik

$\dot{\mathbf{x}} = \mathbf{A}\mathbf{x}+\mathbf{B}u$, $y = \mathbf{C}\mathbf{x}$. Eklem için $\mathbf{x} = (\theta, \dot{\theta})$, $\mathbf{A} = \begin{bmatrix} 0 & 1 \\ 0 & -b/J \end{bmatrix}$, $\mathbf{B} = \begin{bmatrix} 0 \\ 1/J \end{bmatrix}$. **Kontrol edilebilirlik:** $[\mathbf{B}\ \mathbf{AB}\ \cdots\ \mathbf{A}^{n-1}\mathbf{B}]$ tam ranklı ise $u = -\mathbf{K}\mathbf{x}$ ile kapalı çevrim özdeğerleri (kutuplar) *istenen* her yere konabilir.

4.2 Kutup yerleştirme

İstenen karakteristik polinom $\phi(s) = s^2 + a_1 s + a_2$ için Ackermann: $\mathbf{K} = [0\ 1]\,[\mathbf{B}\ \mathbf{AB}]^{-1}\phi(\mathbf{A})$. Elle (2×2): $\det(s\mathbf{I} - \mathbf{A} + \mathbf{BK})$'yı yaz, katsayıları eşitle. LQR (Cilt 4B Bölüm 8) kutupları "maliyet" üzerinden seçer; kutup yerleştirme ise doğrudan. Çok girişli sistemlerde kutup yerleştirme tek değil; LQR daha sistematik.

4.3 Ayrıklaştırma

Dijital kontrolcü Δt'de bir ölçer ve girişi sabit tutar (sıfır dereceli tutucu, ZOH). Kesin ayrık model:

$$\mathbf{A}_d = e^{\mathbf{A}\Delta t}, \qquad \mathbf{B}_d = \int_0^{\Delta t} e^{\mathbf{A}\tau} d\tau\, \mathbf{B},$$

$$\text{pratikte tek seferde:} \quad \exp\left(\begin{bmatrix} \mathbf{A} & \mathbf{B} \\ 0 & 0 \end{bmatrix} \Delta t\right) = \begin{bmatrix} \mathbf{A}_d & \mathbf{B}_d \\ 0 & 1 \end{bmatrix}.$$

Sürekli zamanda tasarlanan $\mathbf{K}$, ayrık uygulamada Δt büyüdükçe bozulur; ayrık kutuplar birim çember içinde olmalı ($|\lambda| < 1$). İdeal ayrık kutup $e^{s\Delta t}$.

Sayılar: kutup yerleştirme ve örnekleme

Kutuplar $-15 \pm 10j$ için $\mathbf{K} = [3.77,\ 0.338]$. Aynı $\mathbf{K}$'yı farklı Δt ile uygulayınca en büyük $|\lambda|$ (ideal $e^{-15\Delta t}$):

Δt	1 ms	10 ms	20 ms	50 ms	80 ms
$\lvert\lambda\rvert$	0.985	0.847	0.685	0.344	**1.546**
ideal	0.985	0.861	0.741	0.472	0.301

1 ms'de sürekli tasarımla aynı; 50 ms'de hâlâ kararlı ama davranış farklı; 80 ms'de kararsız. Kapalı çevrimin doğal frekansı $|s| = 18$ rad/s (≈ 2.9 Hz); 80 ms örnekleme, sinyalin periyodunun neredeyse dörtte biri. Python ve C++ (Taylor serisiyle matris üsteli) aynı sonuç.

4.4 Gözlemci

Sadece θ ölçülüyorsa $\dot{\theta}$'yı **Luenberger gözlemcisi** kestirir: $\dot{\hat{\mathbf{x}}} = \mathbf{A}\hat{\mathbf{x}} + \mathbf{B}u + \mathbf{L}(y - \mathbf{C}\hat{\mathbf{x}})$. Hata dinamiği $\dot{\mathbf{e}} = (\mathbf{A} - \mathbf{LC})\mathbf{e}$; $\mathbf{L}$'yi *dualite* ile kutup yerleştirmeden bul ($(\mathbf{A}^T, \mathbf{C}^T)$ çiftine uygula). Gözlemci kutuplarını kontrol kutuplarının 2–5 katı hızlı seç; çok hızlı seçersen ölçüm gürültüsü büyür (bu ödünleşimi Kalman filtresi en iyi şekilde çözer: Cilt 4B Bölüm 16–17, Cilt 5).

Sayılar: gözlemcili kontrol

Gözlemci kutupları $-60, -70$: $\mathbf{L} = (129.1,\ 4088.7)$. Gerçek başlangıç $\theta = 0.3$, kestirim 0: hız kestirim hatası 14 ms'de 7.2 rad/s tepe yapıyor ("peaking": hızlı gözlemcinin bedeli), 0.1 s'de 0.185, 0.2 s'de 0.005 rad/s; 0.5 s sonra $\theta = 0.0007$ rad. Ayrılma ilkesi (4B Bölüm 16): gözlemci ve kontrolcü ayrı tasarlandı, birlikte kararlı.

Python

```python
import numpy as np
from scipy.linalg import expm
np.set_printoptions(precision=5, suppress=True)

# Eklem: x = [θ, θ˙], ẋ = A x + B u
J, b = 0.0116, 0.01
A = np.array([[0, 1], [0, -b/J]]); B = np.array([[0], [1/J]])

# (1) Kontrol edilebilirlik ve kutup yerleştirme (Ackermann, 2x2)
Co = np.hstack([B, A@B]); print("kontrol edilebilirlik matrisi rank =", np.linalg.matrix_rank(Co))
def acker(A, B, poles):
    a = np.real(np.poly(poles))                                  # istenen karakteristik polinom katsayıları
    phi = A@A + a[1]*A + a[2]*np.eye(2)
    return np.array([[0, 1]])@np.linalg.inv(np.hstack([B, A@B]))@phi
K = acker(A, B, [-15 + 10j, -15 - 10j])
print("istenen kutuplar -15±10j -> K =", K, " kapalı çevrim özdeğerleri:", np.linalg.eigvals(A - B@K))

# (2) Ayrıklaştırma (sıfır dereceli tutucu): A_d = e^{AΔt}, B_d = ∫ e^{Aτ} dτ B  -> blok üstel matris hilesi
def zoh(A, B, dt):
    M = expm(np.block([[A, B], [np.zeros((1, 3))]])*dt); return M[:2, :2], M[:2, 2:]
for dt in [0.001, 0.01, 0.02, 0.05, 0.08]:
    Ad, Bd = zoh(A, B, dt); ev = np.linalg.eigvals(Ad - Bd@K)
    print(f"Δt = {dt*1000:4.0f} ms: sürekli tasarımlı K ayrık uygulanınca |λ| = {np.abs(ev).round(4)} -> {'kararlı' if np.abs(ev).max() < 1 else 'KARARSIZ'}"
          f"  (beklenen e^(-15Δt) = {np.exp(-15*dt):.4f})")

# (3) Gözlemci: sadece θ ölçülüyor, θ˙ kestiriliyor. Luenberger: x̂' = A x̂ + B u + L (y - C x̂)
C = np.array([[1.0, 0.0]]); L = acker(A.T, C.T, [-60, -70]).T                # dualite: (A', C') ile tasarla
print("gözlemci kazancı L =", L.ravel(), " gözlemci özdeğerleri:", np.linalg.eigvals(A - L@C).real)
dt = 0.001; Ad, Bd = zoh(A, B, dt); x = np.array([0.3, 0.0]); xh = np.zeros(2); errs = []
for k in range(500):
    u = -(K@xh)[0]; y = x[0]
    xh = xh + dt*(A@xh + B[:, 0]*u + L[:, 0]*(y - xh[0]))
    x = Ad@x + Bd[:, 0]*u; errs.append(np.abs(x - xh))
errs = np.array(errs)
print(f"gözlemcili kontrol: θ˙kestirim hatasının tepesi {errs[:, 1].max():.2f} rad/s (t ={errs[:, 1].argmax()*dt*1000:.0f} ms); 0.1 s sonra hata {errs[100]}, 0.2 s sonra {errs[200]}; 0.5 s sonra θ = {x[0]:.5f} rad")
```

Çıktı:

```
kontrol edilebilirlik matrisi rank = 2
istenen kutuplar -15±10j -> K = [[3.77  0.338]]  kapalı çevrim özdeğerleri: [-15.+10.j -15.-10.j]
Δt =    1 ms: sürekli tasarımlı K ayrık uygulanınca |λ| = [0.985 0.985] -> kararlı  (beklenen e^(-15Δt) = 0.9851)
Δt =   10 ms: sürekli tasarımlı K ayrık uygulanınca |λ| = [0.847 0.847] -> kararlı  (beklenen e^(-15Δt) = 0.8607)
Δt =   20 ms: sürekli tasarımlı K ayrık uygulanınca |λ| = [0.6851 0.6851] -> kararlı  (beklenen e^(-15Δt) = 0.7408)
Δt =   50 ms: sürekli tasarımlı K ayrık uygulanınca |λ| = [0.3444 0.213 ] -> kararlı  (beklenen e^(-15Δt) = 0.4724)
Δt =   80 ms: sürekli tasarımlı K ayrık uygulanınca |λ| = [0.2106 1.5463] -> KARARSIZ  (beklenen e^(-15Δt) = 0.3012)
gözlemci kazancı L = [ 129.13793 4088.6742 ]  gözlemci özdeğerleri: [-70. -60.]
gözlemcili kontrol: θ̇ kestirim hatasının tepesi 7.17 rad/s (t =14 ms); 0.1 s sonra hata [0.00198 0.18504], 0.2 s
sonra [0.00005 0.00537]; 0.5 s sonra θ = 0.00065 rad
```

C++

```cpp
// e4_zoh.cpp — sıfır dereceli tutucu ile ayrıklaştırma (Taylor serisiyle matris üsteli) ve kararlılık kontrolü
#include <Eigen/Dense>
#include <cmath>
#include <cstdio>
using Mat3 = Eigen::Matrix3d;

Mat3 expm_taylor(const Mat3& M, int terms = 30) {     // küçük, iyi ölçekli matrisler için yeterli
    Mat3 E = Mat3::Identity(), term = Mat3::Identity();
    for (int k = 1; k < terms; ++k) { term = term * M / k; E += term; }
    return E;
}

int main() {
    const double J = 0.0116, b = 0.01;
    Eigen::Matrix2d A; A << 0, 1, 0, -b / J;
    Eigen::Vector2d B(0, 1 / J);
    Eigen::RowVector2d K(3.77, 0.338);                    // Python'daki kutup yerleştirme sonucu (yuvarlanmış)
    for (double dt : {0.001, 0.01, 0.02, 0.05, 0.08}) {
        Mat3 M = Mat3::Zero(); M.topLeftCorner<2, 2>() = A * dt; M.topRightCorner<2, 1>() = B * dt;
        Mat3 E = expm_taylor(M);
        Eigen::Matrix2d Ad = E.topLeftCorner<2, 2>(); Eigen::Vector2d Bd = E.topRightCorner<2, 1>();
        Eigen::Matrix2d Acl = Ad - Bd * K;
        Eigen::Vector2cd ev = Acl.eigenvalues();
        double r = std::max(std::abs(ev(0)), std::abs(ev(1)));
        std::printf("dt =%4.0f ms: en büyük |λ| = %.4f -> %s\n", dt * 1000, r, r < 1 ? "kararlı" : "KARARSIZ");
    }
    return 0;
}
```

Çıktı:

```
dt =   1 ms: en büyük |λ| = 0.9850 -> kararlı
dt =  10 ms: en büyük |λ| = 0.8470 -> kararlı
dt =  20 ms: en büyük |λ| = 0.6851 -> kararlı
dt =  50 ms: en büyük |λ| = 0.3444 -> kararlı
dt =  80 ms: en büyük |λ| = 1.5463 -> KARARSIZ
```

Bölüm 5

Yerçekimi Kompanzasyonu ve Hesaplanmış Tork

Çok eklemli bir robotta eklemler birbirini etkiler: $\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}}+\mathbf{C}\dot{\mathbf{q}}+\mathbf{g}=\boldsymbol{\tau}$ (Cilt 1 Bölüm 6). Her ekleme ayrı PD koymak ("bağımsız eklem kontrolü") basit ama bu kuplajı yok sayar. Modeli kullanan iki adım:

- **PD + yerçekimi kompanzasyonu:** $\boldsymbol{\tau}=\mathbf{K}_p\mathbf{e}+\mathbf{K}_d\dot{\mathbf{e}}+\mathbf{g}(\mathbf{q})$. Statik hatayı sıfırlar (integral terimine gerek kalmaz) ve konum düzenlemede kararlılığı Lyapunov ile ispatlanabilir.
- **Hesaplanmış tork (inverse dynamics control):** $\boldsymbol{\tau}=\mathbf{M}(\mathbf{q})(\ddot{\mathbf{q}}_d+\mathbf{K}_d\dot{\mathbf{e}}+\mathbf{K}_p\mathbf{e})+\mathbf{C}\dot{\mathbf{q}}+\mathbf{g}$. Model doğruysa her eklemin hata dinamiği bağımsız ve doğrusal: $\ddot{\mathbf{e}}+\mathbf{K}_d\dot{\mathbf{e}}+\mathbf{K}_p\mathbf{e}=0$. Kazançlar artık "atalet başına" (Bölüm 1'deki ω_n,ζ doğrudan). Bedeli: modele bağımlılık.

WBC (Cilt 6) hesaplanmış torkun yüzen tabanlı, kısıtlı, çok görevli genellemesidir.

Sayılar: salınım bacağı yörüngesi, 1.25 Hz

Cilt 1'deki 2-uzuvlu bacak; kalça ± 0.5 rad, diz 0.3–1.3 rad. PD: $K_p=6$, $K_d=0.2$; hesaplanmış tork: $\omega_n=15$, $\zeta=1$.

Kontrolcü	RMS hata kalça, diz [°]	max $\lvert\tau\rvert$ [N·m]
PD	2.25, 1.63	1.19
PD + yerçekimi	5.26, 1.53	1.26
Hesaplanmış tork (model doğru)	**0.007, 0.007**	2.89
Hesaplanmış tork (model kütleleri ×1.2)	0.79, 4.14	3.46
Hesaplanmış tork (model ×0.7)	1.76, 9.18	2.02

Sürpriz: yerçekimi kompanzasyonu kalça takibini *kötüleştirdi*. Açıklama: aşağı sarkan bacakta yerçekimi geri çağırıcı bir yay gibi çalışır ve bu hızda harekete yardım eder; onu iptal edince sistemin toplam sertliği düşüyor. Statik tutmada ise tablo net: hedef (0.6, 1.0) rad'da PD (5.08°, 1.87°) kalıcı hata bırakıyor, PD + yerçekimi 0. Elle tahmin: $e\approx\mathbf{g}/K_p=(0.584,\ 0.196)/6$ rad $=(5.6°,\ 1.9°)$. Hesaplanmış tork model doğruyken hatayı 300 kat azaltıyor ama %30 model hatasıyla diz hatası 9°'ye çıkıyor ve PD'den kötü oluyor: **model tabanlı kontrolün değeri modelin doğruluğu kadar** (sistem tanılama: Cilt 4B Bölüm 20).

```python
import numpy as np

# Cilt 1 Bölüm 6'daki 2-uzuvlu bacak (kalça sabit; havadaki salınım bacağı). Model: M(q) q¨ + C q˙ + g = τ
g0 = 9.81; L = np.array([0.2, 0.2]); ms = np.array([0.30, 0.20]); lc = L/2; Is = ms*L**2/12
def M(q):
    c2 = np.cos(q[1]); m11 = Is[0] + Is[1] + ms[0]*lc[0]**2 + ms[1]*(L[0]**2 + lc[1]**2 + 2*L[0]*lc[1]*c2)
    m12 = Is[1] + ms[1]*(lc[1]**2 + L[0]*lc[1]*c2); return np.array([[m11, m12], [m12, Is[1] + ms[1]*lc[1]**2]])
```

```python
def h(q, qd):                                        # C q̇ + g
    hh = -ms[1]*L[0]*lc[1]*np.sin(q[1])
    C = np.array([[hh*qd[1], hh*(qd[0] + qd[1])], [-hh*qd[0], 0.0]])
    g = np.array([ms[0]*g0*lc[0]*np.sin(q[0]) + ms[1]*g0*(L[0]*np.sin(q[0]) + lc[1]*np.sin(q[0] + q[1])), ms[1]*g0*lc[1]*np.sin(q[0] + q[1])])
    return C@qd + g, g

# Hedef: salınım bacağı yörüngesi (kalça ±0.5 rad, diz 0.3..1.3 rad), 1.25 Hz
f = 1.25; w = 2*np.pi*f
def ref(t):
    qd = np.array([0.5*np.sin(w*t), 0.8 - 0.5*np.cos(w*t)])
    qdd = np.array([0.5*w*np.cos(w*t), 0.5*w*np.sin(w*t)]); qddd = np.array([-0.5*w*w*np.sin(w*t), 0.5*w*w*np.cos(w*t)])
    return qd, qdd, qddd
Kp, Kd = np.diag([6.0, 6.0]), np.diag([0.2, 0.2])         # aynı PD kazançları (tork birimi: N·m/rad)
wn = 15.0; Kp_ct, Kd_ct = wn**2*np.eye(2), 2*wn*np.eye(2)    # computed torque: hata dinamiği ë + 2ζω_n ė + ω_n² e = 0

def run(mode, mass_err=1.0, T=4.0, dt=1e-4):
    q = np.array([0.0, 0.3]); qd = np.zeros(2); E = []; tau_max = 0
    for k in range(int(T/dt)):
        t = k*dt; r, rd, rdd = ref(t); e = r - q; ed = rd - qd
        Mh = M(q)*mass_err; hh_, g_ = h(q, qd); hh_ = hh_*mass_err; g_ = g_*mass_err     # kontrolcünün MODELİ (hatalı olabilir)
        if mode == "PD": tau = Kp@e + Kd@ed
        elif mode == "PD+g": tau = Kp@e + Kd@ed + g_
        else: tau = Mh@(rdd + Kd_ct@ed + Kp_ct@e) + hh_                                  # hesaplanmış tork (ters dinamik)
        tau_max = max(tau_max, np.abs(tau).max())
        qdd = np.linalg.solve(M(q), tau - h(q, qd)[0])                                   # GERÇEK sistem
        qd = qd + dt*qdd; q = q + dt*qd
        if t > 2.0: E.append(e)
    E = np.array(E); return np.degrees(np.sqrt(np.mean(E**2, axis=0))), tau_max
print(f"{'kontrolcü':>28} | RMS hata kalça, diz [°] | max |τ| [N·m]")
for mode, me in [("PD", 1.0), ("PD+g", 1.0), ("CT", 1.0), ("CT", 1.2), ("CT", 0.7)]:
    rms, tm = run(mode, me)
    name = {"PD": "PD", "PD+g": "PD + yerçekimi kompanzasyonu", "CT": f"hesaplanmış tork (model×{me})"}[mode]
    print(f"{name:>28} | {rms[0]:6.3f}, {rms[1]:6.3f}            | {tm:.3f}")

# Sabit duruş tutma (statik): hedef q = (0.6, 1.0) rad, 3 s
for mode in ["PD", "PD+g"]:
    q = np.array([0.0, 0.3]); qd = np.zeros(2); target = np.array([0.6, 1.0])
    for k in range(30000):
        e = target - q; tau = Kp@e - Kd@qd + (h(q, np.zeros(2))[1] if mode == "PD+g" else 0)
        qd = qd + 1e-4*np.linalg.solve(M(q), tau - h(q, qd)[0]); q = q + 1e-4*qd
    print(f"statik tutma, {mode:>5}: kalıcı hata {np.degrees(target - q).round(3)} derece")
```

Çıktı:

```
                  kontrolcü | RMS hata kalça, diz [°] | max |τ| [N·m]
                         PD |  2.249,  1.626           | 1.187
PD + yerçekimi kompanzasyonu |  5.256,  1.531           | 1.257
hesaplanmış tork (model×1.0) |  0.007,  0.007           | 2.886
hesaplanmış tork (model×1.2) |  0.793,  4.136           | 3.464
hesaplanmış tork (model×0.7) |  1.759,  9.175           | 2.020
statik tutma,    PD: kalıcı hata [5.075 1.866] derece
statik tutma,  PD+g: kalıcı hata [0. 0.] derece
```

Bölüm 6

Empedans Kontrolü

Konum kontrolü "ne olursa olsun bu açıda kal" der. Bacaklı robot ise sürekli bir çevreyle (yer) temas halindedir: ayak yere değdiği an, sert bir konum kontrolcüsü yerle "kavga eder", büyük darbe kuvvetleri ve sekmeler oluşur. **Empedans kontrolü** bunun yerine bir *davranış* ister: "ayak, sanki k sertliğinde bir yay ve c sönümünde bir amortisör üzerindeymiş gibi davransın":

$$\mathbf{F} = \mathbf{K}(\mathbf{x}_d - \mathbf{x}) + \mathbf{D}(\dot{\mathbf{x}}_d - \dot{\mathbf{x}}), \qquad \boldsymbol{\tau} = \mathbf{J}^T\mathbf{F} \ (+\mathbf{g}(\mathbf{q})).$$

Bu, Cilt 1 Bölüm 4'teki statiği ($\boldsymbol{\tau} = \mathbf{J}^T\mathbf{F}$) bir kontrol yasası olarak kullanmaktır. Tork kontrollü (akım kontrollü, düşük dişli oranlı) aktüatör gerektirir; hobi servolarında doğrudan yapılamaz (Cilt 3 Bölüm 2). **Admitans kontrolü** tersi: kuvveti ölç, sanal yay-sönümden "olması gereken" konumu hesapla, onu konum kontrolcüsüne ver (kuvvet sensörü + sert konum kontrollü robotlarda).

Sayılar: 2 kg'lık tek bacağın 10 cm'den düşmesi

Bacak kütlesiz, aktüatör kuvveti 0–120 N (ağırlığın 6.1 katı) ile sınırlı.

k [N/m]	c [N·s/m]	tepe kuvvet	max sıkışma	1 s sonra (statik denge)
20000	400	120 N (sınırda, 6.1×)	1.95 cm	29.90 cm
5000	150	120 N (sınırda)	1.96 cm	29.61 cm
1500	60	84 N (4.3×)	3.33 cm	28.69 cm
800	40	60 N (3.0×)	5.20 cm	27.55 cm

Sert "konum kontrolü" ($k = 20000$) darbede aktüatörü sınıra dayıyor: gerçek bir motorda bu, akım sınırı, dişli hasarı veya sekme demek. Yumuşak empedans tepe kuvveti yarıya indiriyor, karşılığında bacak daha çok sıkışıyor ve statik duruşta biraz daha alçakta (mg/k kadar) kalıyor. Elle tahmin: çarpma hızı $\sqrt{2gh} = 1.40$ m/s; sönümsüz yayda sıkışma $v\sqrt{m/k}$, $k = 1500$ için 5.1 cm ve kuvvet 77 N (yerçekimi ve sönüm hariç; simülasyon 84 N).

```python
import numpy as np

# Dikey tek bacak: gövde kütlesi m, kütlesiz bacak (boy l0 = 0.30 m), 10 cm'den düşüyor.
# Temas sırasında aktüatör bir "sanal yay-sönüm" uygular: F = k (l0 - l) - c l̇  (empedans kontrolü), F ∈ [0, F_max]
m, g, l0, F_max = 2.0, 9.81, 0.30, 120.0
def drop(k, c, h=0.10, dt=1e-5, T=1.0):
    z = l0 + h; v = 0.0; Fpk = 0.0; zmin = z; contact_t = None; t = 0.0
    for i in range(int(T/dt)):
        if z < l0:                                  # ayak yerde: bacak boyu l = z
            F = np.clip(k*(l0 - z) - c*v, 0, F_max)
            if contact_t is None: contact_t = t
        else: F = 0.0
        a = F/m - g; v += a*dt; z += v*dt; t += dt
        Fpk = max(Fpk, F); zmin = min(zmin, z)
    return Fpk, l0 - zmin, z
print(f"{'k [N/m]':>8} {'c [N·s/m]':>10} | tepe kuvvet   (ağırlığın katı) | max sıkışma | 1 s sonra yükseklik")
for k, c in [(20000, 400), (5000, 150), (1500, 60), (800, 40)]:
```

```
    Fpk, comp, zend = drop(k, c)
    print(f"{k:8.0f} {c:10.0f} | {Fpk:8.1f} N    ({Fpk/(m*g):5.1f}×)          | {comp*100:6.2f} cm  |
    {zend*100:6.2f} cm (statik denge: {(l0 - m*g/k)*100:.2f} cm)")
print(f"aktüatör kuvvet sınırı {F_max} N = ağırlığın {F_max/(m*g):.1f} katı; sert 'konum kontrolü' bu
sınıra dayanıyor")
```

Çıktı:

```
 k [N/m]  c [N·s/m] | tepe kuvvet   (ağırlığın katı) | max sıkışma | 1 s sonra yükseklik
   20000        400 |    120.0 N    (  6.1×)          |   1.95 cm  |  29.90 cm (statik denge: 29.90 cm)
    5000        150 |    120.0 N    (  6.1×)          |   1.96 cm  |  29.61 cm (statik denge: 29.61 cm)
    1500         60 |     84.3 N    (  4.3×)          |   3.33 cm  |  28.69 cm (statik denge: 28.69 cm)
     800         40 |     59.8 N    (  3.0×)          |   5.20 cm  |  27.55 cm (statik denge: 27.55 cm)
aktüatör kuvvet sınırı 120.0 N = ağırlığın 6.1 katı; sert 'konum kontrolü' bu sınıra dayanıyor
```

Bacaklı robotlarda: duruş fazında ayak empedansı (veya WBC'nin kuvvet görevi), salınım fazında sert konum takibi, geçişte kazançların yumuşakça değiştirilmesi. SLIP modeli (Cilt 4B Bölüm 4) tam olarak "bacak bir yaydır" varsayımıdır; empedans kontrolü o yayı yazılımla yaratır.

Bölüm 7

Cilt 4B'ye Geçiş Haritası

Cilt 4B (Underactuated Robotics Çalışma Rehberi) 21 bölüm. Hepsini aynı derinlikte okumak gerekmiyor. Bacaklı/humanoid odağı için öneri:

Öncelik	4B bölümleri	Neden
Çekirdek	2 (sarkaç), 3 (acrobot/cart-pole, PFL), 5 (ZMP, WBC), 7 (DP), 8 (LQR), 9 (Lyapunov), 10 (trajectory optimization), 11 (MPC)	Cilt 6'nın (MPC + WBC yürüyüş) doğrudan önkoşulları
Önemli	4 (yürüme modelleri), 15 (robust, domain randomization), 16 (LQG), 17 (kestirim), 19 (temas)	Gerçek robota geçişte ve sim-to-real'de kritik
İleri	6, 12, 13, 14, 18, 20, 21	Araştırma düzeyi; yüksek lisansta tekrar dönülecek

7.1 4A ile 4B arasındaki köprüler

- 4A Bölüm 1–2 (PD, ikinci derece) → 4B Bölüm 2 (sarkaç): aynı denklem, ama büyük açılarda doğrusal değil.
- 4A Bölüm 4 (kutup yerleştirme) → 4B Bölüm 8 (LQR): kutupları elle seçmek yerine maliyetle.
- 4A Bölüm 3 (faz ve gecikme payı) → 4B Bölüm 15–16 (robustluk, LQG'nin garantisiz payları).
- 4A Bölüm 5 (hesaplanmış tork) → 4B Bölüm 3 (kısmi geri besleme doğrusallaştırması) ve 5 (WBC).
- 4A Bölüm 6 (empedans) → 4B Bölüm 4 (SLIP) ve 19 (temas).

7.2 Staj döneminde 4B ile kâğıt-kalem

4B bölümlerinin her birindeki "Sayılar" kutularını elle yeniden üret. Özellikle:

- Bölüm 8: çift integratör için Riccati denklemini elle çöz ($\mathbf{S} = \begin{bmatrix} \sqrt{3} & 1 \\ 1 & \sqrt{3} \end{bmatrix}$, $\mathbf{K} = [1,\ \sqrt{3}]$ çıkmalı).
- Bölüm 5: LIPM'de ZMP formülü ve capture point'i elle türet (Kitapçık 1 B7.4 hazırlık).
- Bölüm 9: sarkaç için enerji tabanlı Lyapunov fonksiyonunun türevini elle al; LaSalle'ı uygula.
- Bölüm 7: 3 durumlu küçük bir problemde değer iterasyonunun 3 adımını elle yap.

Bölüm 8

Sorular

Kâğıt, kalem, hesap makinesi. **(K)**: kod okuma.

E1 — İkinci dereceden sistemler

E1.1 $J = 0.01$, $K_p = 4$, $K_d = 0.2$: ω_n, ζ, aşım ve yaklaşık yerleşme süresi?

E1.2 $J = 0.01$ için $\omega_n = 10$ rad/s, $\zeta = 0.7$ veren K_p, K_d?

E1.3 Kutuplar $-3 \pm 4j$. ω_n ve ζ nedir? Aşım?

E2 — PID

E2.1 Yerçekimi torku 0.559 N·m olan bir eklem, $K_p = 8$ ile PD kontrollü. Kalıcı hata yaklaşık kaç radyan/derece?

E2.2 İntegral birikmesini kendi cümlelerinle açıkla; kırpma (clamp) kuralı neyi kontrol eder?

E2.3 Türev filtresi zaman sabiti 5 ms, döngü 1 kHz: filtre katsayısı $a = \tau_f/(\tau_f + \Delta t)$?

E2.4 Türev hatadan alınıyor, $K_d = 0.4$, $\Delta t = 1$ ms, hedef 0'dan 1 rad'a basamakla değişiyor. İlk adımda türev teriminin katkısı? Türev ölçümden alınsaydı?

E2.5 (K) Basit PID: `e = r - y; u = kp*e + i; i + = ki*e*dt` ($K_d = 0$, doyma yok). $k_p = 2$, $k_i = 1$, $\Delta t = 0.1$, $r = 1$. İlk çağrıda $y = 0$, ikincide $y = 0.5$. Her çağrıda u ve çağrı sonrası i?

E3 — Frekans tanım

E3.1 (zor) $L(s) = \frac{4}{s(s+2)}$. Kazanç kesişim frekansını, faz payını ve gecikme payını hesapla.

E3.2 $\omega_c = 35$ rad/s olan bir döngüye 20 ms gecikme ekleniyor. Faz payından kaç derece gider?

E3.3 Kapalı çevrim bant genişliği 5 Hz isteniyor. Kontrolcü en az hangi frekansta çalışmalı (kural)? Bir hobi servosunun 50 Hz komut hızı bu iş için yeter mi?

E4 — Durum uzayı

E4.1 $\mathbf{A} = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}$. $\mathbf{B} = (0,1)^T$ ve $\mathbf{B} = (1,0)^T$ için kontrol edilebilirliği incele; ikincisinin fiziksel anlamı?

E4.2 Aynı $\mathbf{A}$, $\mathbf{B} = (0,1)^T$: kutupları -2 ve -3'e koyan $\mathbf{K} = [k_1,\ k_2]$?

E4.3 Çift integratörün ZOH ile ayrık modeli $\mathbf{A}_d$, $\mathbf{B}_d$'yi yaz.

E4.4 Gözlemci kutupları neden kontrol kutuplarından hızlı seçilir; çok hızlı seçmenin bedeli nedir?

E5 — Hesaplanmış tork

E5.1 $\boldsymbol{\tau} = \mathbf{M}(\ddot{\mathbf{q}}_d + \mathbf{K}_d\dot{\mathbf{e}} + \mathbf{K}_p\mathbf{e}) + \mathbf{h}$ uygulandığında, model kesin doğruysa hata denkleminin $\ddot{\mathbf{e}} + \mathbf{K}_d\dot{\mathbf{e}} + \mathbf{K}_p\mathbf{e} = 0$ olduğunu göster.

E5.2 2-uzuvlu bacakta $\mathbf{q} = (0.6,\ 1.0)$ rad'da $\mathbf{g}(\mathbf{q})$'yu hesapla ($m_1 = 0.3$, $m_2 = 0.2$, $\ell_1 = 0.2$, $\ell_c = 0.1$). $K_p = 6$ ile PD'nin kalıcı hatasını tahmin et.

E5.3 Asılı bir salınım bacağında yerçekimi kompanzasyonu takip hatasını neden artırabilir? Ters sarkaç gibi dik duran bir gövdede durum neden tersine döner?

E6 — Empedans

E6.1 $m = 2$ kg, sanal yay $k = 1500$ N/m: statik sıkışma, doğal frekans, kritik sönüm katsayısı; $c = 60$ ise ζ?

E6.2 10 cm'den düşen bu kütlenin çarpma hızı? Sönümsüz yayda (yerçekimini ihmal ederek) en büyük sıkışma ve kuvvet?

E6.3 Neden yürüyen robotlar duruş fazında sert konum kontrolü yerine empedans kullanır? Bir hobi servosu ile empedans kontrolü neden zordur?

Bölüm 9

Cevaplar

E1

E1.1 $\omega_n = \sqrt{4/0.01} = 20$ rad/s; $\zeta = 0.2/(2\sqrt{4 \cdot 0.01}) = 0.5$; aşım $e^{-\pi 0.5/\sqrt{0.75}} = 16.3\%$; $t_s \approx 4/(0.5 \cdot 20) = 0.4$ s.

E1.2 $K_p = J\omega_n^2 = 1$, $K_d = 2\zeta\omega_n J = 0.14$.

E1.3 $\omega_n = \sqrt{9+16} = 5$, $\zeta\omega_n = 3 \Rightarrow \zeta = 0.6$; aşım $e^{-\pi 0.6/0.8} = 9.5\%$.

E2

E2.1 $e \approx 0.559/8 = 0.070$ rad $= 4.0^\circ$ (simülasyon 0.068).

E2.2 Aktüatör sınırdayken sistem daha hızlı gidemez ama hata büyük kaldığı için integral büyümeye devam eder; hedefe ulaşıldığında bu "birikmiş" integral torku hedefi aşmaya zorlar. Kırpma: çıkış doymadaysa *ve* hata doymayı derinleştiren yöndeyse integrali güncelleme.

E2.3 $a = 0.005/0.006 = 0.833$.

E2.4 $\dot{e} \approx (1-0)/0.001 = 1000$ rad/s, $K_d\dot{e} = 400$ N·m: anlık dev bir tekme (doymaya gider). Ölçümden: y değişmediği için 0.

E2.5 1. çağrı: $e = 1$, $u = 2 \cdot 1 + 0 = 2$, sonra $i = 0.1$. 2. çağrı: $e = 0.5$, $u = 1 + 0.1 = 1.1$, sonra $i = 0.15$.

E3

E3.1 $|L| = \frac{4}{\omega\sqrt{\omega^2+4}} = 1 \Rightarrow \omega^4 + 4\omega^2 - 16 = 0 \Rightarrow \omega^2 = -2 + \sqrt{20} = 2.472$, $\omega_c = 1.572$ rad/s. $\angle L = -90^\circ - \arctan(\omega_c/2) = -128.2^\circ$, $PM = 51.8^\circ$. Gecikme payı $= 0.904/1.572 = 0.575$ s.

E3.2 $35 \cdot 0.02 = 0.7$ rad $= 40.1^\circ$.

E3.3 10–20 kat: 50–100 Hz. 50 Hz sınırda; üstelik servo 20 ms'ye kadar gecikme ekler (E3.2'deki gibi büyük faz kaybı). 5 Hz bant genişliği hobi servosuyla gerçekçi değil; 1–2 Hz daha makul.

E4

E4.1 $\mathbf{B} = (0,1)$: $[\mathbf{B}\ \mathbf{AB}] = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$, rank 2, kontrol edilebilir. $\mathbf{B} = (1,0)$: $\begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}$, rank 1: giriş konumu doğrudan değiştiriyor ama hızı hiç etkilemiyor; hız kendi başına sabit kalır.

E4.2 $\det(s\mathbf{I} - \mathbf{A} + \mathbf{BK}) = s^2 + k_2 s + k_1$; istenen $(s+2)(s+3) = s^2 + 5s + 6 \Rightarrow \mathbf{K} = [6,\ 5]$.

E4.3 $\mathbf{A}_d = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix}$, $\mathbf{B}_d = \begin{bmatrix} \Delta t^2/2 \\ \Delta t \end{bmatrix}$ (sabit ivmeyle kinematik).

E4.4 Kontrolcü kestirime dayandığı için kestirim, kontrol edilen dinamikten hızlı oturmalı. Çok hızlı (büyük $\mathbf{L}$): ölçüm gürültüsü kestirime büyütülerek girer ve başlangıçta büyük "tepe" hataları olur (Bölüm 4: 7.2 rad/s).

E5

E5.1 Gerçek sistem: $\mathbf{M}\ddot{\mathbf{q}} + \mathbf{h} = \boldsymbol{\tau} = \mathbf{M}(\ddot{\mathbf{q}}_d + \mathbf{K}_d\dot{\mathbf{e}} + \mathbf{K}_p\mathbf{e}) + \mathbf{h} \Rightarrow \mathbf{M}(\ddot{\mathbf{q}} - \ddot{\mathbf{q}}_d - \mathbf{K}_d\dot{\mathbf{e}} - \mathbf{K}_p\mathbf{e}) = 0$. $\mathbf{M}$ tersinir, $\ddot{\mathbf{e}} = \ddot{\mathbf{q}}_d - \ddot{\mathbf{q}}$: $\ddot{\mathbf{e}} + \mathbf{K}_d\dot{\mathbf{e}} + \mathbf{K}_p\mathbf{e} = 0$.

E5.2 $g_1 = 0.3 \cdot 9.81 \cdot 0.1 \sin 0.6 + 0.2 \cdot 9.81(0.2 \sin 0.6 + 0.1 \sin 1.6) = 0.584$ N·m; $g_2 = 0.2 \cdot 9.81 \cdot 0.1 \sin 1.6 = 0.196$. Kalıcı hata $\approx g/K_p = (0.097,\ 0.033)$ rad $= (5.6^\circ,\ 1.9^\circ)$ (simülasyon 5.1°, 1.9°).

E5.3 Asılı bacakta yerçekimi geri çağırıcı bir yay: sistemi sertleştirir; iptal edince toplam sertlik düşer, takip kötüleşebilir. Dik gövdede (ters sarkaç) yerçekimi kararsızlaştırıcı ("negatif yay"): kompanze edilmezse PD'nin sertliğinin bir kısmını yer, hatta yetersiz kazançta düşürür.

E6

E6.1 $mg/k = 1.31$ cm; $\omega_n = \sqrt{1500/2} = 27.4$ rad/s; $c_{kr} = 2\sqrt{km} = 109.5$; $\zeta = 60/109.5 = 0.55$.

E6.2 $v = \sqrt{2 \cdot 9.81 \cdot 0.1} = 1.40$ m/s; enerji: $\frac{1}{2}mv^2 = \frac{1}{2}kx^2 \Rightarrow x = v\sqrt{m/k} = 5.1$ cm; $F = kx = 77$ N.

E6.3 Yer ile temas belirsiz zamanlı ve sert; empedans darbe kuvvetini sınırlar, zemin düzensizliklerine uyum sağlar, sekmeyi azaltır. Hobi servosu tork değil *konum* komutu alır, torku ölçemez ve dişlisi geri sürülemez: istenen yay davranışını üretemez (en fazla konum komutunu yavaşça yumuşatabilirsin).